SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

CSA04 – OPERATING SYSTEMS

ASSIGNMENT 1 – CO1

Evaluate and Justify the Operating-System Design of a Smart Hospital

Item	Details
Student name	MUKESH V
Register number	192524514
Course	Operating Systems (CSA04)
Course outcome	CO1 – Operating-system types, structures and system calls
Total marks	50 (Tasks 1–11: 42 marks; Task 12 Day-4 Design Challenge: 8 marks)
Bloom's level	K5 – Evaluate
Nature of assignment	Individual, one week
Submission date	6 November 2026
GitHub repository link	https://github.com/mukeshdinesh2008-lgtm/Slot-A-Operating-system-

Contents

Task	Title	Marks
–	Execution details and GitHub repository	–
1	Evaluate the Requirements of Each Computing Environment	4
2	Judge the Type of Operating System for Each Environment	3
3	Evaluate Operating-System Structures	4
4	Evaluate the Services an Operating System Must Provide	4
5	Examine System Calls	4
6	Evaluate the Cost of System Calls	4
7	Evaluate Process Control System Calls	4
8	Evaluate CPU States and Protection	3
9	Evaluate the Memory Hierarchy and I/O Systems	4
10	Measure and Compare	4
11	Recommend and Defend the Operating-System Plan	4
12	Re-evaluate and Adapt the Design (Day-4 Design Challenge)	8
	Total	50

Execution Details and GitHub Repository

Every program in this assignment was executed individually on Linux, and the outputs, commands and source files are kept in the GitHub repository named above, with one folder for each task. The repository README.md contains the details in the table below.

Table 0.1 – Execution environment

Item	Details
Operating system	Ubuntu 24.04.5 LTS (Linux kernel 6.18), virtual machine with 1 CPU core
Compiler	gcc 13.3.0
Compile command	gcc -std=c11 -Wall -Wextra -pthread -o <name> <name>.c
System-call tracing	strace 6.8 (strace -c and strace -e trace=…)
Timing	clock_gettime(CLOCK_MONOTONIC) inside the programs and the shell time command

Table 0.2 – Repository folders, programs and how to run them

Folder	Files	How to run	Result
Task05	rd.c, strace_trace.txt, strace_c.txt	gcc … rd.c -o rd; strace -e trace=openat,read,write,close ./rd	4 read() calls for 10,000 fgetc() calls
Task06	copy.c, output.txt, strace_c.txt	gcc … copy.c -o copy; ./copy; strace -c ./copy	10001 / 21 / 4 read() calls (matches expected)
Task07	shell.c, shell_variant.c, output.txt, variant_out.txt	gcc … shell.c -o shell; ./shell; ./shell_variant > variant_out.txt	Output matches expected; duplicate lines explained
Task10	spawn.c, spawn2.c, mem.c, bigcopy.c and output files	gcc … -pthread file.c; ./a.out	Tables 6.1, 7.1, 10.2
Tasks 1–4, 8, 9, 11, 12	Report sections and tables (README notes)	Not programs – design and evaluation	Tables in this report

Commit plan: at least one commit for each task (for example ‘Task 6: completed copy.c and strace output’), committed progressively as each task is finished.

TASK 1 – Evaluate the Requirements of Each Computing Environment

The hospital contains six computing environments: bedside patient monitors, nurses' handheld tablets, doctors' desktop workstations, the central database server, the MRI imaging server and the link to the partner hospitals. Each environment is evaluated against five requirements: response time, reliability, throughput, battery life and security. Rank 1 means the most important requirement for that environment.

1.1 Ranked Requirement Table

Table 1.1 – Ranking of requirements (1 = most important, 5 = least important)

Environment	Response	Reliability	Throughput	Battery	Security	Justification of the ranking
Bedside monitor	1	2	5	4	3	An alarm must sound within 10 ms, so a missed deadline is a patient-safety failure. A monitor that hangs is the next worst failure. It handles one patient's few sensor streams, so throughput is unimportant. It is mains powered with a battery backup.
Nurse tablet	3	4	5	1	2	The tablet must last a full shift, so battery is first. It holds patient data and can be lost or stolen, so security is second. A delay of a second is acceptable.
Doctor workstation	2	3	4	5	1	Doctors view and edit full records, so access control comes first. The interface must feel immediate. It is mains powered, so battery is irrelevant.
Database server	4	1	3	5	2	Records 'must never stop', so reliability is first, followed by confidentiality. It serves hundreds of clients, so throughput matters more than latency of one query.
Imaging (MRI) server	4	3	1	5	2	MRI processing is a large batch workload, so total work per hour (throughput) dominates. Images are sensitive patient data.
Partner-hospital link	4	2	3	5	1	Records cross an external network, so authentication and encryption are first. A link that drops loses shared care information.

1.2 Conflicting Requirements

Conflict	Explanation	Resolution
Fast response vs many users on one machine	A time-sharing scheduler divides the CPU among n tasks. The wait for one task can reach n × quantum, which can exceed 10 ms.	Do not share monitors with other work; use a priority-based real-time scheduler.
Throughput vs response time	Large batch jobs (MRI) keep the CPU busy but delay interactive requests.	Separate servers: MRI on its own multiprocessor server.
Battery life vs performance	Higher clock speed and always-on services drain the battery.	Handheld OS with power management and a reduced service set.
Security vs response time	Encryption, authentication and logging add processing time.	Apply heavy security at the server and the link, a light authenticated channel on monitors.
Reliability vs update frequency	Security patches normally need restarts, but the database must never stop.	Cluster with rolling updates: patch one node while the other serves.

1.3 Requirements Critical to Patient Safety

- **Bounded alarm latency (monitors):** a late alarm can cost a life. This is a hard real-time requirement.
- **Database availability and integrity:** doctors and nurses make drug and treatment decisions from the records. A stopped or corrupted database is a clinical risk.
- **Protection of records:** wrong or leaked patient data harms the patient, so isolation between users is safety-critical.
- **Tablet battery over a shift:** a dead tablet during a drug round delays treatment, so it is a secondary safety requirement.

1.4 Lessons from the History of Operating Systems

Generation	Lesson	Where it applies in the hospital
Batch systems	Maximise CPU utilisation by queuing long jobs and running them without interaction.	MRI image processing runs as queued jobs on the imaging server.
Time-sharing	Share one machine fairly among many interactive users, at the cost of unpredictable delay for each.	Database server and workstations serve many users. This is the model that must not be used for monitors.

Generation	Lesson	Where it applies in the hospital
Personal computers	One user, a friendly interface, little need for protection in early versions.	Doctor workstations. Modern versions add the multi-user protection that early PCs lacked.
Mobile and handheld	Limited battery, memory and screen; apps are suspended when not in use.	Nurse tablets need power management and a reduced service set.
Real-time and embedded	Correctness depends on meeting deadlines; the OS is small and predictable.	Bedside monitors.

1.5 Alternative Concept and Complexity Analysis – One General-Purpose OS Everywhere

If the IT team's proposal were adopted, the same time-sharing general-purpose OS (for example a desktop Linux or Windows) would run on every device. The solution would be a single image, one set of tools and one patch cycle for the whole hospital. The consequences are evaluated below.

Table 1.2 – Per-environment OS versus one general-purpose OS (values are typical figures)

Criterion	Different OS per environment (proposed)	One general-purpose OS everywhere
Worst-case alarm response	Hard real-time RTOS with priority pre-emption: bounded and constant, O(1) scheduling decision, typically well under 1 ms.	Fair time-sharing scheduler: with n runnable tasks and quantum q the wait can reach about n × q. With 20 tasks and q = 4 ms this is about 80 ms, which breaks the 10 ms deadline. There is no upper bound if a driver or the memory manager blocks the CPU.
Memory needed per monitor	RTOS kernel: tens of KB to a few MB.	Full OS: hundreds of MB to over 1 GB, which forces costlier hardware on every bed.
Administration effort	Several images, tools and skills to maintain (higher effort).	One image and one patch process (lower routine effort). However, every patch can force a restart of safety-critical devices.
Battery on tablets	Handheld OS with power management.	Desktop services run in the background, so the battery does not last a shift.
Time complexity (alarm path)	O(1), deterministic.	O(n) in the number of runnable tasks, unbounded in the worst case.
Space complexity	O(K), K = small kernel.	O(K′), K′ much larger, with every service loaded.

Conclusion: one general-purpose OS reduces administration effort but cannot guarantee the 10 ms alarm deadline, and wastes memory and battery. The safety-critical requirement decides the matter, so different OS types are justified.

Evidence: *Table 1.1 (ranked requirements with justification), Table 1.2 and the conflict table above.*

TASK 2 – Judge the Type of Operating System for Each Environment

2.1 Comparison of Operating-System Types

Type	Characteristics	Suitability for the hospital
Mainframe	Batch and transaction processing, very high I/O capacity and reliability.	Style of the database server (reliability, transaction throughput), but a full mainframe is too costly.
Desktop	Single user, interactive, time-sharing scheduler, rich GUI.	Doctor workstations.
Multiprocessor	Several CPUs share memory and one OS (SMP) or have roles assigned (AMP).	Imaging server.
Distributed	Independent computers cooperate over a network, appearing as one system.	Link to the partner hospitals and the later cloud record service.
Clustered	Several nodes share storage and monitor each other with fail-over.	Database server.
Hard real-time	A missed deadline is a system failure; bounded latency.	Bedside monitors.
Soft real-time	Deadlines are important but an occasional miss is tolerable.	Not for alarms; possible for display refresh or video from the imaging server.
Handheld	Small battery, touch screen, restricted services and aggressive power saving.	Nurse tablets.

2.2 Chosen Type for Each Environment

Table 2.1 – Chosen OS type, with rejected alternatives and reasons

Environment	Chosen type	Rejected alternatives and reasons
Bedside monitors	Hard real-time OS (FreeRTOS or QNX)	Desktop or server OS: no bounded latency. Soft real-time: a missed alarm deadline is unacceptable.
Nurse tablets	Handheld (mobile) OS	Desktop OS: heavy services, short battery life, slower start-up.
Doctor workstations	Desktop (multi-user time-sharing) OS	Handheld OS: screen and input too limited. Real-time OS: no need for deadlines.
Database server	Clustered server OS (Linux) with fail-over	Single large server: single point of failure. Mainframe: cost.
Imaging server	Symmetric multiprocessing (SMP) server OS	Asymmetric multiprocessing: master processor becomes a bottleneck. Cluster: image processing needs shared memory.

Environment	Chosen type	Rejected alternatives and reasons
Partner link	Distributed system over an encrypted VPN/TLS channel	Open unencrypted link: unacceptable security risk. Tightly coupled cluster: the partners are separate organisations.

2.3 Symmetric versus Asymmetric Multiprocessing for the Imaging Server

Aspect	Symmetric (SMP)	Asymmetric (AMP)
Organisation	All processors run the same OS and are peers; any CPU can run any task.	A master CPU runs the OS and assigns work to slave CPUs.
Load balancing	Automatic, by one shared ready queue or per-CPU queues.	Depends on the master; slaves may be idle.
Bottleneck	Shared memory bus and lock contention.	The master CPU.
Failure	Remaining CPUs continue.	Master failure stops the system.
Verdict	**Chosen** for MRI processing, because the work divides evenly across the processors.	Rejected.

2.4 Clustered Database Server with Fail-over

Two or more nodes share storage (or replicate it). A heartbeat signal passes between the nodes. If the active node stops responding, the standby takes over its IP address and mounts the database within seconds. Clients reconnect and continue. Rolling maintenance is also possible: patch the standby, fail over, then patch the former active node. The database therefore never has to be stopped for an update.

2.5 Alternative Concept and Complexity Analysis – Single Large Multiprocessor Instead of a Cluster

If the database ran on one large multiprocessor machine, all work would go to one box with many CPUs and a large shared memory. Availability, cost and throughput are compared below.

Criterion	Cluster with fail-over	Single large multiprocessor
Availability	If one node is available a fraction a of the time, two nodes give $A = 1 - (1 - a)^2$. With $a = 0.999$, $A = 0.999999$ (about 32 seconds of downtime a year in theory), plus the fail-over time of a few seconds per failure.	$A = a = 0.999$, about 8.8 hours of downtime a year. The machine is a single point of failure, and updates need a restart.
Cost	Several mid-range servers; cost grows about linearly with nodes.	Very large SMP machines cost disproportionately more per CPU.

Criterion	Cluster with fail-over	Single large multiprocessor
Throughput with N processors (time)	Scales by adding nodes, but data must be replicated or partitioned.	Amdahl's law: speed-up $S = 1 / ((1 - p) + p/N)$. With $p = 0.95$: $N = 8$ gives $S \approx 5.9$, $N = 32$ gives $S \approx 12.5$, and S can never exceed 20. Lock contention limits scaling further.
Memory (space)	Memory of each node is separate: total memory is the sum, but each node holds only its share or a replica. Space for data is $O(R \times D)$ with R replicas.	One large shared memory (for example terabytes) is directly addressable by all CPUs; data is stored once, O(D).

Conclusion: for a database that must never stop, availability outweighs the simplicity of one large machine, so the cluster is chosen.

Evidence: *Tables 2.1 and the SMP/AMP and cluster comparisons above.*

TASK 3 – Evaluate Operating-System Structures

3.1 Comparison of Structures

Table 3.1 – Operating-system structures

Structure	Description	Reliability	Performance	Ease of updating
Simple (monolithic)	The whole OS runs as one program in kernel mode; any part can call any other directly.	Low: a fault in any driver can crash the kernel.	High: a service is a plain function call.	Hard: usually needs a rebuild and reboot.
Layered	The OS is split into layers, each using only the layer below.	Medium: errors are easier to locate.	Lower: every request passes through several layers.	Medium.
Microkernel	Only scheduling, memory and IPC stay in the kernel. Drivers, file systems and networking are user-space servers.	High: a failing driver can be restarted without a reboot.	Lower: services need message passing.	Easy: restart or replace a server.
Modular (loadable kernel modules)	A monolithic core that loads drivers and file systems as modules.	Medium (modules still run in kernel mode).	High, near monolithic.	Good: load or unload a module without reboot.
Hybrid	A mixture, usually a microkernel-like core with many services in kernel space.	Medium to high.	Medium to high.	Medium.

3.2 Structure of Real Operating Systems

System	Structure	Notes
Linux	Monolithic with loadable kernel modules	Whole kernel in kernel mode, drivers and file systems loaded as modules (modprobe).
Windows	Hybrid	A microkernel-influenced executive with many services in kernel mode; subsystems in user mode.
Android	Layered system on a modified Linux kernel	Linux kernel, hardware abstraction layer, native libraries and runtime, application framework, apps.
FreeRTOS	Minimal kernel library linked with the application	Scheduler, queues and timers only; no memory protection on most ports.
QNX	Microkernel	A small kernel plus user-space drivers and services; message passing is the main service interface.

3.3 Structure Chosen for Two Environments

Environment	Chosen structure	Defence
Patient monitors	Microkernel RTOS (QNX-style) or a very small real-time kernel	A faulty sensor or display driver must not stop the alarm task. The kernel is small enough to analyse for worst-case timing. Services restart without a reboot.
Database server	Monolithic kernel with loadable modules (Linux)	The highest throughput for file and network I/O, mature drivers, and modules can be updated without a reboot. Reliability is obtained at cluster level by fail-over.

3.4 Alternative Concept and Complexity Analysis – Monolithic Kernel on the Monitors

If the bedside monitors used a monolithic kernel, drivers and services would be called directly inside one address space. A request would cost one system call instead of messages between servers.

Criterion	Microkernel (chosen)	Monolithic kernel
Time cost of one service request	A request is a message to a server and a reply: at least two context switches and message copying, typically a few microseconds. Cost O(1) but with a larger constant.	One system call (trap and return), typically a fraction of a microsecond. Cost O(1) with a smaller constant.
Kernel memory	Kernel of tens of KB, with servers as separate processes.	Kernel of several MB or more, all drivers resident in kernel memory.
Effect of a faulty driver	Only that driver process fails; it is restarted and the alarm task continues.	A driver bug can overwrite kernel data and crash the entire monitor (kernel panic).
Predictability	Small kernel with bounded worst-case paths.	Large non-pre-emptible code paths make worst-case latency harder to bound.

Conclusion: the extra microseconds of message passing are far below the 10 ms alarm deadline, so the microkernel's fault isolation is worth the cost on the monitors.

Evidence: *Table 3.1 and the defended choices in section 3.3.*

TASK 4 – Evaluate the Services an Operating System Must Provide

4.1 Service Table for Each Environment

Table 4.1 – Operating-system services per environment (Yes / Reduced / No)

Service	Monitor	Tablet	Desktop	DB server	Imaging	Justification
User interface	No	Yes	Yes	No	No	A GUI uses memory and CPU, so it is removed on the monitor (fixed display) and the servers (remote shell or batch).
Program execution	Yes	Yes	Yes	Yes	Yes	Every system must load and run programs; the monitor runs only a few fixed tasks.
I/O operations	Yes	Yes	Yes	Yes	Yes	Essential everywhere (sensors and display on the monitor, DMA on the imaging server).
File system	Reduced	Yes	Yes	Yes	Yes	The monitor needs only a small flash store; the database server needs a journaling file system.
Communication	Reduced	Yes	Yes	Yes	Yes	A lean secure network stack is enough on the monitor.
Error detection	Yes	Yes	Yes	Yes	Yes	Detecting a failed sensor is itself a safety function (self-test, watchdog); ECC and logs on servers.
Resource allocation	Reduced	Yes	Yes	Yes	Yes	Fixed priorities and static memory suffice on the monitor.
Accounting	No	Reduced	Reduced	Yes	Reduced	Audit trails of record access are needed on the database server.
Protection and security	Yes	Yes	Yes	Yes	Yes	Signed firmware on monitors, encryption and remote wipe on tablets, full access control on servers.

4.2 Services Removed from the Bedside Monitor

The monitor is made smaller and faster by removing the graphical user interface, multi-user login and accounting, the general-purpose file system (replaced by a small flash store), dynamic loading of programs, virtual memory and swapping (a page fault would break the deadline), and unused network protocols. Fewer services mean less code to be tested, less memory, a shorter boot time and a smaller attack surface.

4.3 Services Essential for the Database Server

The database server needs a journaling file system (consistency after a crash), efficient I/O and buffering, process and thread management for many clients, memory protection and resource allocation (so one query cannot starve the others), strong authentication, access control and accounting (audit logs of who read which record), and error detection and recovery (ECC memory, RAID, log monitoring, fail-over).

4.4 Protection of One Patient's Records from Another User's Programs

- **Authentication:** each user logs in with a unique identity (password plus a second factor).
- **Authorisation:** file permissions or access-control lists allow only the treating team to read a patient's records.
- **Memory protection:** the hardware MMU gives each process its own address space, so a program cannot read another program's memory.
- **User and kernel mode:** programs run in user mode and can reach files and devices only through system calls that the OS checks.
- **Encryption and audit:** records are encrypted at rest and in transit, and every access is logged.

4.5 Alternative Concept and Complexity Analysis – Full Desktop OS on the Tablets

Criterion	Reduced handheld OS	Full desktop OS with every service
Memory and storage (space)	Typically 1–2 GB of RAM in use and a few GB of storage for the system.	Typically 3–4 GB of RAM in use and 20 GB or more of storage for the system, with many unused services resident. Space grows with the number of services, O(S).
Start-up time (time)	Typically seconds to tens of seconds.	Typically a minute or more, because more services start and the file system is larger.
Battery life	Services are suspended when idle; a shift of 8–12 hours is achievable.	Background services and a higher power draw can shorten this to a few hours, which would not last a shift.
Security	Smaller attack surface and sandboxed apps.	Larger attack surface.

The figures are typical orders of magnitude, not measurements made on a hospital tablet. The reduced handheld OS is therefore retained.

Evidence: *Table 4.1 with the justified inclusions and exclusions.*

TASK 5 – Examine System Calls

5.1 Categories of System Calls

Category	Purpose	Examples (Linux)
Process control	Create, run, wait for and end processes; allocate memory.	fork, execve, wait, exit, brk, mmap
File management	Create, open, read, write, close and delete files.	open, read, write, close, unlink
Device management	Request, use and release devices.	ioctl, read, write on device files
Information maintenance	Get or set system data such as time, process attributes.	getpid, gettimeofday, uname
Communication	Pipes, shared memory, messages and sockets.	pipe, socket, send, recv, shmget
Protection	Control access to resources.	chmod, chown, umask, setuid

5.2 What Happens When a Program Calls read()

- **1. Library call:** the program calls the C library function read(fd, buf, n). The library places the system-call number and the arguments in the registers defined by the architecture (on x86-64 Linux: rax = call number, rdi, rsi, rdx = arguments).
- **2. Trap:** the library executes a special instruction (syscall on x86-64). This is a software interrupt that transfers control to a fixed kernel entry point.
- **3. Mode switch:** the hardware changes the mode bit from user mode to kernel mode, saves the program counter and registers, and switches to the kernel stack.
- **4. Kernel work:** the kernel looks up the call number in the system-call table, checks that the file descriptor and the buffer address are valid, checks permissions, finds the data in the page cache (or starts a disk read and puts the process in the waiting state), and copies the data into the user buffer.
- **5. Return:** the kernel stores the return value (number of bytes read, or −1 with errno) in a register, restores the saved registers, switches back to user mode and the library function returns to the program.

5.3 Passing Parameters to the Kernel

Method	How it works	Used by
Registers	Parameters are placed directly in CPU registers. Fast, but limited by the number of registers.	Linux x86-64 (up to six parameters).
Block in memory	The parameters are stored in a block or table in memory and its address is placed in a register. Allows any number of parameters.	Linux when there are more than five parameters (on some architectures), Solaris.
Stack	The program pushes the parameters on the stack and the OS pops them.	Some older systems and 32-bit conventions.

5.4 Evidence from strace

The program rd.c (Task05/rd.c) writes a 10,000-byte file, reads it back with read() using a 4096-byte buffer, and then reads it again with fgetc(). It was compiled with gcc -std=c11 -Wall -Wextra and traced with strace -e trace=openat,read,write,close ./rd. The relevant lines (the dynamic-loader lines are omitted) are shown below.

```
openat(AT_FDCWD, "t.dat", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
write(3, "ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEF"..., 4096) = 4096
write(3, "OPQRSTUVWXYZABCDEFGHIJKLMNOPQRST"..., 4096) = 4096
write(3, "CDEFGHIJKLMNOPQRSTUVWXYZABCDEFGH"..., 1808) = 1808
close(3)                                = 0
openat(AT_FDCWD, "t.dat", O_RDONLY)     = 3         <- direct read()
read(3, "ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEF"..., 4096) = 4096
read(3, "OPQRSTUVWXYZABCDEFGHIJKLMNOPQRST"..., 4096) = 4096
read(3, "CDEFGHIJKLMNOPQRSTUVWXYZABCDEFGH"..., 4096) = 1808
read(3, "", 4096)                       = 0         <- end of file
close(3)                                = 0
openat(AT_FDCWD, "t.dat", O_RDONLY)     = 3         <- fgetc() loop, 10,000 calls
read(3, "ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEF"..., 4096) = 4096
read(3, "OPQRSTUVWXYZABCDEFGHIJKLMNOPQRST"..., 4096) = 4096
read(3, "CDEFGHIJKLMNOPQRSTUVWXYZABCDEFGH"..., 4096) = 1808
read(3, "", 4096)                       = 0
```

Figure 5.1 – strace output of rd.c (Task05/strace_trace.txt)

The output shows that fgetc() was called 10,000 times but caused only four read() system calls, each with a 4096-byte request, because the C library buffers the file. The write() calls were also made in 4096-byte units by the library.

5.5 Alternative Concept and Complexity Analysis – fopen/fread/fwrite versus open/read/write

If the programs used fopen, fread and fwrite, the library would keep a buffer for each FILE and call read() or write() only when the buffer is empty or full. The code would be fread(buf, 1, n, fp) instead of read(fd, buf, n).

Criterion	Direct open/read/write	Library fopen/fread/fwrite
System calls for a file of n bytes	n / b calls for a program buffer of b bytes. With b = 1 this is n calls.	About n / 4096 calls whatever size of request the program makes (measured: 10,000 fgetc() calls produced 4 read() calls).
Time complexity	O(n / b) system calls, each costing much more than a function call.	O(n / 4096) system calls plus n cheap library calls in user mode.
Extra space	None beyond the program's own buffer, O(b).	One library buffer per open file, typically 4 KB, O(B) per FILE.
Control	Exact control of when data reaches the kernel.	Data may stay in the buffer until fflush, fclose or exit (a risk if the program crashes).

Evidence: *system-call table, step-by-step description and strace output above (Task05 folder in the repository).*

TASK 6 – Evaluate the Cost of System Calls

6.1 Why a System Call Costs More than a Function Call

An ordinary function call only pushes a return address and jumps, taking a few nanoseconds. A system call must execute a trap, switch from user to kernel mode, save and restore registers, switch stacks, validate the arguments, run the kernel code and return. It may also disturb the CPU caches and branch predictor, and the data must be copied between kernel and user space. A system call therefore costs hundreds of nanoseconds to microseconds, which is tens to hundreds of times more than a function call. For this reason a backup program should make as few system calls as possible.

6.2 Completed Code

TODO 1 was completed with a loop that reads up to bufsize bytes, counts every read(), stops when read() returns 0 or −1 and writes exactly the n bytes that were read. The rest of the program was not changed.

```
long calls = 0;
ssize_t n;
while (1) {
    n = read(in, buf, bufsize);
    calls++;                    /* count every read() call */
    if (n <= 0) break;          /* 0 = end of file, -1 = error */
    if (write(out, buf, (size_t)n) != n) { calls = -1; break; }
}
```

Figure 6.1 – Code that replaces TODO 1 (Task06/copy.c)

The program was compiled with gcc -std=c11 -Wall -Wextra -o copy copy.c (no warnings) and run on Ubuntu 24.04.5 LTS with gcc 13.3.0. The output is identical to the expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Figure 6.2 – Program output

The counts follow from the formula ceil(n / b) + 1, where the extra call is the final read() that returns 0. For n = 10,000: b = 1 gives 10,000 + 1; b = 512 gives 20 + 1; b = 4096 gives 3 + 1.

6.3 strace -c Comparison

The program was run with strace -c -e trace=read,write,openat ./copy. The summary was:

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 50.99    0.005958           0     10027           write
```

```
 48.46    0.005663           0     10027           read
  0.55    0.000064           7         9           openat
100.00    0.011685           0     20063           total
```

Figure 6.3 – strace -c summary (Task06/strace_c.txt)

The read() count of 10,027 matches the program's output: 10,001 + 21 + 4 = 10,026 calls from copy_file(), plus one read() made by the dynamic loader to read the ELF header of the C library. The write() count of 10,027 is 10,000 + 20 + 3 = 10,023 from copy_file(), plus 3 writes from the stdio calls that created the test file, plus 1 write for the printed output (stdout was a pipe, so it was flushed once).

6.4 Measurements for Different Buffer Sizes

To see the time effect, a second program (Task10/bigcopy.c) copied a 10 MB file with each buffer size. The times were measured with clock_gettime on a Linux virtual machine with one CPU core.

Table 6.1 – Copying a 10 MB file with read() and write()

Buffer size (bytes)	read() calls	Time (ms)	Speed-up compared with 1 byte
1	10,485,760	5,209.1	1×
64	163,840	82.2	≈ 63×
512	20,480	28.0	≈ 186×
4,096	2,560	14.1	≈ 369×
65,536	160	9.2	≈ 566×
1,048,576	10	8.5	≈ 613×

The number of calls falls in proportion to 1/b and the time falls sharply up to about 4 KB. Beyond 64 KB the time barely improves (9.2 ms to 8.5 ms) because the cost is no longer dominated by system-call overhead but by moving the data itself. A very large buffer also uses more memory and gives no benefit.

6.5 How Library Buffering Reduces System Calls

With fread and fwrite the C library fills a buffer of about 4096 bytes with one read() and then serves the program's small requests from memory. Task 5 showed this: 10,000 fgetc() calls resulted in only four read() system calls.

6.6 Recommendation for the Hospital Backup Program

The recommended buffer is **64 KB (65,536 bytes)**, a multiple of the 4 KB page size. It needs only 160 system calls for 10 MB, is within about 8% of the time of a 1 MB buffer (9.2 ms against 8.5 ms) and uses little memory. The 1-byte and 64-byte buffers are unacceptable for a database backup.

6.7 Alternative Concept and Complexity Analysis – Memory-Mapped Files (mmap)

With mmap the backup program maps the source file into its address space and the destination as well, and then copies memory to memory. The pseudocode is:

```
src = open(srcfile, O_RDONLY);  fstat(src, &st);
dst = open(dstfile, O_RDWR|O_CREAT|O_TRUNC, 0644);  ftruncate(dst, st.st_size);
s = mmap(NULL, size, PROT_READ,  MAP_PRIVATE, src, 0);
d = mmap(NULL, size, PROT_WRITE, MAP_SHARED,  dst, 0);
memcpy(d, s, size);   // page faults load pages on demand
munmap(s, size); munmap(d, size); close(src); close(dst);
```

Figure 6.4 – Pseudocode of the mmap copy

Criterion	read()/write() with a buffer	mmap
Number of system calls	About n / b: 2,560 for 10 MB with a 4 KB buffer.	A few (open, fstat, mmap, munmap, close), independent of n: O(1).
Copying between kernel and user space	Each byte is copied from the page cache to the user buffer and back to the kernel: two copies.	The page-cache pages are mapped directly into the program, so there is no kernel-to-user copy; only the final memcpy remains.
Page faults	None caused by the buffer.	About n / 4096 minor page faults as pages are touched, so the cost moves from system calls to page faults.
Memory for very large files	Fixed, O(b), whatever the file size.	The virtual address space needed is O(n), though only the touched pages are resident. On a 32-bit system a file larger than the address space cannot be mapped at once.

Conclusion: mmap is attractive for random access to large files, but a plain 64 KB read/write loop is simpler, uses constant memory and was already within a small margin of the best measured time, so it is kept for the backup program.

Evidence: *Tables 6.1, program output, strace -c summary (Task06 folder).*

TASK 7 – Evaluate Process Control System Calls

7.1 What fork(), exec(), wait() and exit() Do

Call	Function
fork()	Creates a child process that is a copy of the parent. It returns 0 in the child and the child's process ID in the parent (−1 on error). The address space is copied lazily (copy-on-write).
exec() family	Replaces the program in the calling process with a new program. The process ID stays the same. It returns only if it fails.
wait() / waitpid()	Blocks the parent until a child ends, and collects its exit status. This also removes the child's entry from the process table.
exit() / _exit()	Ends the process, closes its files, releases its memory and returns an exit status to the parent. exit() flushes the stdio buffers first, _exit() does not.

7.2 How a Shell Runs a Command and Obtains the Exit Status

The shell reads the command line, calls fork(), and the child calls exec() to become the requested program. The parent calls waitpid(). When the program calls exit(status), the kernel stores the status in the child's entry in the process table and wakes the parent. waitpid() returns it, and the macro WEXITSTATUS(status) extracts the low 8 bits, which the shell reports (for example in $?).

Figure 7.1 – Process diagram of the shell running job 1 (expr 6 * 7)

Parent (shell), PID 100		Child, PID 101		Kernel actions
printf, fflush				
fork()	→	child created (copy of parent)		new PCB, copy-on-write pages
waitpid(101) – parent waits		execlp("expr", ...) – program replaced		load expr, same PID 101
blocked		prints 42, exit(0)	→	state = zombie, status 0 saved
waitpid returns, WEXITSTATUS = 0	←			child removed from process table

7.3 Zombie and Orphan Processes

Type	Meaning	How the OS deals with it
Zombie	A child that has ended but whose parent has not yet called wait(). Its process-table entry remains to hold the exit status.	The entry is removed when the parent calls wait(). If the parent never does, the zombie stays until the parent exits, so servers must always reap their children (or ignore SIGCHLD).

Type	Meaning	How the OS deals with it
Orphan	A child whose parent ended first.	The init process (PID 1, or a sub-reaper) adopts it and calls wait() when it ends, so no zombie is left.

7.4 Why fork() Followed by exec() Instead of One 'Create Process' Call

Separating the two steps lets the child change its environment between fork() and exec(): it can redirect standard input and output, set up pipes, change directory, drop privileges or set resource limits. A single create-process call would need a very large number of parameters for all of these. The separation is flexible and keeps each call simple. The cost of copying the parent is small because of copy-on-write.

7.5 Completed Code and Output

TODO 1 was completed as follows (Task07/shell.c). The variables status and pid, which the given code used without declaring, were declared at the top of main.

```
pid = fork();
if (pid == 0) {
    execlp("expr", "expr", "6", "*", "7", (char *)NULL);
    exit(127);                  /* reached only if exec fails */
}
waitpid(pid, &status, 0);
printf("job 1 exit status %d\n", WEXITSTATUS(status));
```

Figure 7.2 – Code that replaces TODO 1

Compiled with gcc -std=c11 -Wall -Wextra and run on Linux, the output matched the expected output exactly:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Figure 7.3 – Program output

7.6 Why stdout Is Flushed before Each fork()

When stdout is redirected to a file or a pipe, the C library buffers it fully. fork() copies the parent's memory, including any text still waiting in the buffer, so both processes would later write the same buffered text. Calling fflush(stdout) before fork() empties the buffer so that the child starts with an empty buffer.

7.7 Experiment: Removing the Second fflush and Using exit(3)

The second fflush(stdout) was removed, _exit(3) was changed to exit(3), and the output was redirected to a file (./shell_variant > variant_out.txt). The file contained:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Figure 7.4 – Content of variant_out.txt

Because the output is redirected to a file, stdout is fully buffered, and the text ‘job 1 exit status 0’ and ‘shell: running job 2’ was still in the buffer when the second fork() was made. The child received a copy of that buffer. Because the child now ended with exit(3), which flushes the stdio buffers (unlike _exit), it wrote its copy of the buffer to the file. The parent wrote the same text again when it ended. Therefore ‘shell: running job 2’ (and the line before it) appears twice. On a terminal this does not show, because stdout is line buffered and is flushed at each newline.

7.8 Alternative Concept and Complexity Analysis – posix_spawn() or Threads

With posix_spawn() the call creates the new process and loads the program in one step, normally without copying the parent’s address space (it uses vfork-like semantics). With threads the command would run inside the same process, so it could not be a separate program unless it called fork() anyway.

Table 7.1 – Measured time to start 1,000 commands (/bin/true), 3 runs each

Method	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)
fork() + execl() + waitpid()	904.2	706.5	684.3
posix_spawn() + waitpid()	647.9	671.9	672.6
fork() + exit only (no exec)	50.1	47.2	47.4
pthread_create() + join	8.9	8.2	8.9

Criterion	fork() + exec()	posix_spawn()	Threads
Time to start 1,000 commands	About 0.7 ms per command (measured 684–904 ms per 1,000).	About 0.65–0.67 ms per command, a little faster and more stable.	About 0.009 ms, but they cannot run a separate program.

Criterion	fork() + exec()	posix_spawn()	Threads
Memory copied or shared	Page tables copied, pages shared copy-on-write. Cost grows with the parent's size, O(m) pages.	Not copied, O(1) in the size of the parent.	Everything shared, only a new stack.
Isolation between commands	Full: separate address spaces.	Full.	None: one faulty command can crash all.

Total time for n commands is O(n) in all cases, with different constants. The measurements show that exec and process teardown dominate the cost (about 0.7 ms) and that the saving from posix_spawn is modest on this machine. The shell therefore keeps fork() and exec() for its flexibility and isolation.

Evidence: *working program and output (Task07 folder), process diagram and measured table above.*

TASK 8 – Evaluate CPU States and Protection

8.1 User Mode, Kernel Mode, Privileged Instructions and the Mode Bit

The CPU has a mode bit. In kernel mode (mode bit 0) the OS can execute every instruction and access all memory and devices. In user mode (mode bit 1) programs cannot execute privileged instructions. Privileged instructions include I/O instructions, enabling or disabling interrupts, loading the memory-management registers, changing the timer and halting the CPU. A user program that attempts one causes a trap to the OS, which normally terminates the program. The mode changes to kernel mode only through a system call, an interrupt or an exception, and returns to user mode when the OS gives control back, so a user program can never run with kernel privileges.

8.2 Interrupts, Traps and the Timer Interrupt

Event	Cause	Example in the hospital
Interrupt (hardware)	An asynchronous signal from a device. The CPU saves its state and runs an interrupt service routine.	The sensor sends an abnormal reading; the network card receives a packet.
Trap (software)	A synchronous event caused by the program: a system call, or an exception such as division by zero or an invalid address.	A program calls read(); a program touches memory that is not its own.
Timer interrupt	A hardware timer interrupts the CPU at fixed intervals so that the OS regains control.	On workstations and servers the scheduler uses it to pre-empt a program that has used its time slice. On the monitor it drives the periodic scheduler tick.

8.3 What Would Happen if Any Program Could Disable Interrupts

A buggy or malicious program could disable interrupts and never enable them again. The sensor interrupt would then never be handled, so no alarm would be raised for an abnormal reading. The timer interrupt would stop as well, so the OS could never pre-empt the program and the whole monitor would hang in a loop. This is why disabling interrupts is a privileged instruction available only in kernel mode.

8.4 How the Hardware Stops One Program from Reading Another's Memory

The memory-management unit (MMU) translates every address that a program produces. In a simple scheme, a base register and a limit register define the legal range, and any address outside it causes a trap. In paging, each process has its own page table, and a program can reach

only the frames mapped in its table. Entries also carry read, write and execute permission bits. The OS loads the base and limit (or page-table) registers with privileged instructions during a context switch, so a user program cannot change them. An illegal access raises a trap and the OS terminates the process (a segmentation fault on Linux).

8.5 Alternative Concept and Complexity Analysis – Polling Instead of Interrupts

With polling, a loop on the monitor repeatedly reads the sensor register and tests for an abnormal value, for example: while (1) { v = read_sensor(); if (v > LIMIT) alarm(); wait(T); }.

Criterion	Interrupt-driven	Polling
Worst-case delay before an abnormal reading is noticed	The interrupt latency, typically microseconds, and bounded.	The polling interval T plus the processing time. For T = 10 ms, the delay can reach the full 10 ms deadline before processing even starts; to meet the deadline T must be much shorter, such as 1 ms.
CPU time wasted	None while no event occurs; the CPU runs other tasks or sleeps.	Every poll costs CPU time even when nothing happens. If each poll takes 5 µs and T = 1 ms, 0.5% of the CPU is spent on polling; with T = 100 µs it is 5%.
Power	The CPU can enter a low-power sleep state between events.	The CPU wakes at every poll, increasing power use and battery drain.
Time complexity	O(1) per event.	O(1) per poll but a poll happens every T whether or not there is an event, so the cost is proportional to time elapsed, O(t / T).

Conclusion: interrupts are required on the monitors. Polling is acceptable only for very short waits or very high event rates.

Evidence: *reasoned explanation of CPU modes, interrupts and protection applied to the monitors.*

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems

9.1 The Memory Hierarchy

Table 9.1 – Levels of the memory hierarchy (typical figures)

Level	Typical access time	Typical size	Hospital use
Registers	< 1 ns	Under 1 KB	Values being computed.
Cache (L1–L3)	1–20 ns	32 KB – tens of MB	Hot code and data automatically held by the CPU.
Main memory (DRAM)	50–100 ns	GB to TB	Latest monitor readings; database buffer pool; MRI image being processed.
SSD	50–150 µs	100s of GB to TB	Current patient records, database indexes and logs.
Hard disk	5–10 ms	TB	Older patient records and archived MRI images.
Network storage / cloud	1 ms to 100+ ms	Practically unlimited	Backups and the shared records of partner hospitals.

9.2 Data-Placement Plan

Data	Where kept	Reason
Monitor readings	Main memory (a circular buffer) on the monitor, with a short log in flash; summaries sent to the database.	Needed within milliseconds; very small; the loss of old samples is acceptable.
Patient records (current)	SSD, with a buffer cache in the server's main memory; replicated to the standby node.	Frequent random access with low latency and consistency.
Patient records (older)	Hard disk or network storage.	Rarely read, so a lower cost per GB is more important.
MRI images (being processed)	Main memory, streamed from the scanner by DMA.	Large, sequential, and processed immediately.
MRI images (archive)	Hard-disk arrays and network backup.	Huge volume, rarely read.

9.3 Programmed I/O, Interrupt-Driven I/O and DMA

Method	How it works	CPU involvement	Suitable for
Programmed I/O	The CPU repeatedly tests the device status and moves each word itself.	Constant: the CPU waits and copies.	Very simple, fast devices.

Method	How it works	CPU involvement	Suitable for
Interrupt-driven I/O	The device interrupts the CPU when it is ready; the CPU then moves a word.	One interrupt per word or small block.	Slow devices such as sensors and keyboards.
DMA	The DMA controller moves a whole block between the device and memory, and interrupts once at the end.	Only to start the transfer and to handle the final interrupt.	**MRI scanner**, disks, network cards.

9.4 Effect of Caching on Record Look-up

A cache keeps recently used records in a faster level. If the hit ratio is h, the effective access time is h × t_cache + (1 − h) × t_slow. For example, with a buffer cache in memory (100 ns) and SSD (100 µs), a hit ratio of 99% gives 0.99 × 0.1 µs + 0.01 × 100 µs = 1.1 µs on average, about 90 times faster than going to the SSD each time. With a hit ratio of 90% it is 10.1 µs. Because a small set of patients (those in the hospital now) accounts for most look-ups, hit ratios are high.

9.5 Alternative Concept and Complexity Analysis – Interrupt-Driven I/O for the MRI Scanner

Assumptions: a 500 MB image (500 × 1,048,576 = 524,288,000 bytes), a 4-byte word, 2 µs of CPU time to service one interrupt, and for DMA a transfer of 1 MB blocks with the same 2 µs per interrupt. These are illustrative values.

Criterion	Interrupt per word	DMA
Number of interrupts	524,288,000 / 4 = 131,072,000.	524,288,000 / 1,048,576 = 500.
CPU time for interrupts	131,072,000 × 2 µs ≈ 262 seconds, which is more than the time to transfer the data, leaving the CPU unable to do other work.	500 × 2 µs = 1 ms.
Time complexity	O(n) interrupts for n words.	O(n / B) interrupts for block size B.
Memory buffers needed	One word at a time in a register or small buffer, O(1), then copied by the CPU into memory.	A buffer of at least one block (1 MB), or the full 500 MB destination, O(B), written directly by the controller.

Conclusion: DMA is chosen for the MRI scanner. Interrupt-driven I/O is kept for the slow sensors.

Evidence: *Table 9.1, the data-placement plan and the I/O method comparison.*

TASK 10 – Measure and Compare

All measurements were made on one Linux system: Ubuntu 24.04.5 LTS, Linux kernel 6.18, gcc 13.3.0, a virtual machine with one CPU core. Results will differ on other machines, so the trends, not the absolute values, are the important finding.

10.1 System Calls for Different Buffer Sizes (strace -c)

Table 10.1 – Number of read() calls for the 10,000-byte file

Buffer size (bytes)	read() calls (program)	Expected	Match
1	10,001	10,001	Yes
512	21	21	Yes
4096	4	4	Yes

strace -c reported 10,027 read() calls in total (10,026 from the program plus one by the dynamic loader), which agrees with the table (see Task 6).

10.2 Time in User Mode and Kernel Mode

The 10 MB copy program (bigcopy), which runs all six buffer sizes, was run under the shell's time command:

```
real    0m5.414s
user    0m1.208s
sys     0m4.104s
```

Figure 10.1 – time ./bigcopy

Kernel (sys) time is about 76% of the total. Almost all of it was spent in the 1-byte case (5.2 s of the 5.4 s of wall-clock time in Table 6.1), where 10 million read() and 10 million write() calls were made. This confirms that the cost of a system call is dominated by the work of entering and leaving the kernel, not by the amount of data.

10.3 Creating 1,000 Processes and 1,000 Threads

Table 10.2 – Time to create and finish 1,000 processes (fork) and 1,000 threads (pthread_create)

Method	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)
1,000 × fork() + waitpid()	50.1	47.2	47.4
1,000 × pthread_create() + pthread_join()	8.9	8.2	8.9

Creating a thread was on average about 5.5 times faster than creating a process (about 8.7 ms against 48.2 ms for 1,000). A separate program (Task10/mem.c) created 1,000 threads that were all alive at the same time and read VmRSS from /proc/self/status: resident memory rose from 1,396 kB to 10,228 kB, an increase of 8,832 kB, or about 8.8 kB per thread. Each thread also reserves a large virtual stack (8 MB by default), but only the pages that are touched count as resident.

10.4 Do the Measurements Support the Earlier Judgements?

Earlier judgement	Measurement	Verdict
A system call is expensive, so the number of calls must be small (Tasks 5 and 6).	1-byte buffer: 5,209 ms; 4 KB: 14.1 ms; 64 KB: 9.2 ms for the same 10 MB.	Supported strongly.
Library buffering cuts the number of system calls (Task 5).	10,000 fgetc() calls produced 4 read() calls.	Supported.
Threads are cheaper to create than processes (Task 7).	8.7 ms against 48.2 ms for 1,000.	Supported (about 5.5×).
posix_spawn is faster than fork + exec (Task 7).	648–672 ms against 684–904 ms per 1,000, but with small and overlapping gaps.	Partly supported: the difference is small here.
A very large buffer is not needed (Task 6).	64 KB (9.2 ms) is close to 1 MB (8.5 ms).	Supported.

10.5 Alternative Concept and Complexity Analysis – Threads Instead of Processes

The measurements in Table 10.2 already compare the two. A thread shares the code, data, heap and open files of its process and owns only its stack, registers and thread-local data. A new process needs its own process control block, page tables and file-descriptor table; the pages are shared copy-on-write until a write occurs.

Criterion	1,000 processes	1,000 threads
Creation time (measured)	≈ 48 ms (about 48 µs each).	≈ 8.7 ms (about 8.7 µs each).
Memory	Own page tables, kernel structures and (after writing) own copies of pages: the larger cost per unit.	About 8.8 kB resident per thread (measured), plus a reserved virtual stack.
Explanation	The kernel must duplicate the memory-management and file structures.	The kernel creates only a new schedulable entity and a stack; address space and files are shared.
Isolation	Strong.	None; one thread can corrupt the others' data.

Criterion	1,000 processes	1,000 threads
Complexity	O(n) creation time; O(n × per-process overhead) space.	O(n) creation time with a smaller constant; O(n × stack) space.

Evidence: *Tables 10.1 and 10.2, the time output and the memory measurement. The source files and output files are in the Task10 folder.*

TASK 11 – Recommend and Defend the Operating-System Plan

11.1 Recommended Plan

Table 11.1 – Complete operating-system plan for the hospital

Environment	OS type	Structure	Services	Protection	I/O method
Bedside monitors	Hard real-time (QNX/FreeRTOS)	Microkernel or minimal RTOS kernel	Sensor I/O, alarm task, secure link, watchdog; no GUI, no accounting, no swap	Signed firmware, authenticated link, mode separation	Interrupt-driven
Nurse tablets	Handheld OS	Layered on a Linux kernel	Touch UI, Wi-Fi, secure app sandbox	Disk encryption, remote wipe, per-app sandbox	Interrupt-driven, DMA for storage
Doctor workstations	Desktop OS	Monolithic with modules or hybrid	Full GUI, printing, file services	Multi-user login, ACLs, MMU isolation	Interrupt-driven and DMA
Database server	Clustered server OS with fail-over	Monolithic with loadable modules (Linux)	Journaling file system, multi-user, audit, rolling updates	Authentication, ACLs, encryption, audit log	DMA, buffered with page cache
Imaging server	SMP server OS	Monolithic with modules	Batch job scheduling, large memory	Encrypted storage, role-based access	DMA
Partner link	Distributed system	Server OS with network services	VPN or TLS gateway, replication	Mutual authentication, encryption, firewall	Interrupt-driven, DMA on the network card

11.2 Evidence from the Analysis and Measurements

- **Alarm latency (Tasks 1, 3, 8):** only a real-time kernel with interrupt handling can bound the delay; a time-sharing wait grows with n × quantum.
- **System-call cost (Tasks 5, 6, 10):** copying 10 MB took 5,209 ms with a 1-byte buffer and 9.2 ms with a 64 KB buffer, so the backup and database software use buffered I/O of 64 KB or more.
- **Process and thread cost (Tasks 7, 10):** a process costs about 48 µs to create and a thread about 8.7 µs, so the database server uses a thread pool for its many clients and keeps separate processes for isolation of different services.
- **Availability (Task 2):** a two-node cluster improves a 99.9% available node to a theoretical 99.9999%.
- **Interrupts and DMA (Tasks 8, 9):** DMA reduces the CPU time for a 500 MB image from about 262 s to about 1 ms of interrupt handling in the illustrative calculation.

11.3 Limitations and Risks

- Five or six OS families increase the skills and tools needed for administration and the patching effort.
- A real-time OS gives less choice of drivers and software than a general-purpose OS.
- Fail-over has a short interruption (seconds), and a fault of the shared storage can still stop both nodes unless the data is also replicated.
- A fault in a shared network or the VPN would stop the partner link.
- My measurements were taken on a virtual machine with one CPU, so the absolute figures will differ on hospital hardware and the real-time latency of the monitors has to be measured on the target device.

11.4 Suitability for a Group of Three Hospitals

The plan remains suitable. The monitors, tablets and workstations are per-hospital and scale linearly, since a new hospital adds its own copies. The clustered database can be extended with more nodes and with replication between hospitals, and the partner link already uses a distributed design. The points that need to be added are a central identity and access-management service shared by the three hospitals, a consistent data-replication policy and a central monitoring service.

11.5 Alternative Concept and Complexity Analysis – Virtual Machines on Two Large Servers

Criterion	Separate physical machines (plan)	All systems as VMs on two large servers
Response time of the monitors	Hard real-time on dedicated hardware: bounded.	The hypervisor and other VMs share the CPU, adding a variable scheduling delay and network latency; the 10 ms bound can no longer be guaranteed. Monitors should therefore stay on their own hardware.
Memory and CPU overhead	None extra.	Each VM needs its own guest OS (hundreds of MB to GB), and virtualisation typically costs a few percent of CPU on CPU-bound work and more on I/O. Space O(V × guest OS) for V VMs.

Criterion	Separate physical machines (plan)	All systems as VMs on two large servers
Reliability	A hardware fault affects one environment only.	A server fault stops all VMs on it, unless VMs are live-migrated or restarted on the second server. This improves consolidation but concentrates the risk.
Management	Many devices.	Fewer machines, snapshots and easier back-up.

Conclusion: virtualising the servers (database, imaging, workstations' desktop images) on two hosts is acceptable and reduces cost, but the patient monitors must stay on dedicated real-time hardware.

Evidence: *Table 11.1 and the measurements of Tasks 5–10.*

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Changed requirement: the hospital adds remote monitoring devices in patients' homes that send readings over the internet, and moves patient records to a cloud service shared with the partner hospitals. **Decision: MODIFY** – the original design is kept for the hospital, with new components added and some existing ones changed.

A. Impact Analysis

Table 12.1 – Effect of the changed requirements

Area	Impact	Affected?
OS type for monitors	Home devices need the same bounded local alarm, but run outside the protected hospital network and on a patient's power supply and internet.	New embedded real-time OS type needed for home devices; hospital monitors unchanged.
OS type for records	The single clustered database in the hospital becomes a cloud, distributed and replicated service shared by three organisations.	Database server and partner link are redesigned as a distributed system.
Services	Added: secure remote communication, over-the-air updates, store-and-forward buffering, identity management. Reduced: the local file system on the database server shrinks to a cache.	Yes.
Protection and security	The attack surface grows from one building to the whole internet and the cloud provider. Records are now outside the hospital's own server room.	Strongly affected.
Response time	Internet latency is variable (tens to hundreds of ms) and cannot be bounded.	The alarm path must not depend on the internet.
Reliability	Home internet or power can fail; the cloud may be unreachable.	Local operation and buffering are required.

B. Evaluation of the Original Design

Criterion	Can the original plan support the change?	Evidence
Response time	Partly. The RTOS on a home device meets the local alarm deadline, but an alarm sent to the hospital over the internet cannot meet 10 ms.	Interrupt-driven alarm is local (Task 8); network delay is unbounded.

Criterion	Can the original plan support the change?	Evidence
Reliability	No. The plan has no store-and-forward and no behaviour for loss of connection. The on-site cluster does not cover a cloud outage or a failed home link.	Fail-over analysis in Task 2 covers only on-site nodes.
Security	No. The plan assumed devices inside a controlled building, with security by VPN to only two partners. Home devices on household routers and a cloud service need device identity, end-to-end encryption and strict access control.	Task 4 protection mechanisms apply on a trusted LAN.
Scalability	Partly. The cluster scales for the hospital, but thousands of home devices and three hospitals need elastic cloud capacity.	Task 11 limits and risks.

Conclusion: the plan can be kept for the in-hospital environments, but cannot support home devices and cloud records without changes, so the decision is to modify rather than retain or fully redesign it.

C. Adaptation: the Revised Plan

Table 12.2 – Revised operating-system plan

Component	Change	Reason
Hospital bedside monitors	Retained (hard real-time RTOS).	Their requirements have not changed.
Home monitoring devices	New: embedded real-time OS (FreeRTOS or similar) with a local alarm on the device, TLS to the cloud, secure boot, signed over-the-air updates and a buffer that stores readings during an internet failure.	The alarm stays local and bounded; the internet only carries reports.
Patient-records service	Modified: a distributed, replicated database in the cloud (at least three replicas in separate zones) with strong consistency for clinical records, plus an on-site cluster kept as a cache and a fall-back for the hospital.	Shared by three hospitals and available even if one site fails.
Partner link	Replaced by the cloud service and identity federation.	One shared service replaces point-to-point links.
Gateway	New: an edge gateway at each hospital (and optionally at home) that authenticates devices, filters and buffers data.	Protects core systems from the internet.

Component	Change	Reason
Security	Per-device certificates, mutual TLS, role-based access control, encryption at rest, audit logs and compliance with the applicable data-protection law.	The attack surface is larger.
Tablets and workstations	Retained, but they access records through the cloud API and keep a cache for use in the event of a link failure.	Same needs, new data location.

D. Evidence-Based Defence

Why modify and not retain: the original design cannot meet the reliability and security requirements outside the hospital (section B). **Why modify and not redesign:** the in-hospital requirements are unchanged. The measurements of Tasks 5–10 on system-call cost and process cost still apply to the servers, and the real-time arguments of Tasks 1, 3 and 8 still apply to the bedside and home devices. Redesigning everything would add cost and risk without benefit.

Table 12.3 – Trade-offs of the decision

Decision	Gain	Cost or risk
Alarm kept on the device	Bounded response time independent of the internet (Task 8: interrupt latency in microseconds).	A more capable (more costly) home device is needed.
Cloud, replicated records	Availability across hospitals; no new on-site servers when scaling to three hospitals.	Dependence on a provider and the internet, higher latency than a local database (milliseconds instead of microseconds, so local caching is added), and data-protection duties.
Strong consistency for records	Doctors always see the latest record.	Slower writes and lower availability during a network split than eventual consistency.
Store-and-forward at home	No loss of readings during an outage.	Alarm reports to the hospital may arrive late; the patient must be told whether the link is down.
Keeping the on-site cluster as a cache	The hospital continues to operate during a cloud outage.	Extra synchronisation complexity.

Final decision: MODIFY. Keep the hospital design, add embedded real-time devices with local alarms for patients' homes, and move the records to a replicated, secured cloud service with an on-site cache. The remaining risks (the cloud provider, the internet and device security) are managed with redundancy, encryption, signed updates and monitoring.

Evidence: *Tables 12.1–12.3 and the references to Tasks 1–11.*